Informe de módulo — M6 (parte base de datos)

1. Identificación

- **Módulo:** M6 Comunidad, interfaz y base de datos — **solo la parte base de datos** (la interfaz fue aplazada por acuerdo del equipo).
- **Responsable de esta entrega:** Samir Yesid Iguaran Torres. (El documento general asigna el modelo de datos de M6 a Luis Felipe Cotes Castrillo; coordinar quién firma esta parte.)
- **Fecha:** 26 de octubre de 2026 · **Versión del informe:** 0.1
- **Destinatarios:** Líder del proyecto/compañero que lo pidió y Diego Maldonado (M1).

2. Requerimientos cubiertos

Código	Estado
RF-22 Favoritos / lista personal	Capa de datos y seguridad hecha. Interfaz **pendiente**.
RF-23 Valoraciones y comentarios	Capa de datos y seguridad hecha. Interfaz **pendiente**.
RF-25 Panel del autor (vistas y valoraciones)	Datos y función `panel_autor()` hechos. Interfaz **pendiente**.
RNF-02 Seguridad (acceso por rol)	RLS en todas las tablas de M6.
RNF-04 Compatibilidad navegador/móvil	**Pendiente** (pertenece a la interfaz).

3. Qué se hizo

Cuatro migraciones SQL para Supabase/PostgreSQL (carpeta `supabase/migrations/`), cada una con su reversa en `supabase/migrations_down/`.

Decisiones (todas PROPUESTAS, aún no aprobadas por todo el equipo):

1. Lista personal de estudio = favoritos en la v1 (el PDF no pide funciones distintas).
2. Valoración 1–5, una por usuario y simulación, editable. Las filas individuales son privadas; el público solo ve promedio y total.
3. Comentarios: texto plano de 1 a 1000 caracteres (tras recortar espacios). Sin DELETE físico: borrado lógico (`eliminado_en`) por el autor o un moderador. Solo el autor edita el contenido. El escape de HTML se hace al mostrar (la BD guarda el texto tal cual).
4. Vista válida: un visitante cuenta una vez por simulación y día (UTC). El navegador genera un token aleatorio; solo se guarda su hash SHA-256 (sin datos personales). Solo se cuentan simulaciones publicadas.
5. El panel del autor es una **función** (`panel_autor()`), no una vista, porque la tabla de vistas no es legible directamente.

4. Interfaces con otros módulos

M6 ofrece

- Tablas: `favoritos`, `valoraciones`, `comentarios`, `vistas_simulacion`.
- Vista: `valoraciones_resumen(simulacion_id, promedio, total)` (lectura pública).
- Funciones: `registrar_vista(p_simulacion_id uuid, p_token text) → boolean` (anon y authenticated); `panel_autor() → (simulacion_id, estado, vistas, promedio, total_valoraciones)` (authenticated); `simulacion_es_publica(uuid) → boolean`.

M6 espera (PROVISIONAL, por acordar)

- **M1 (Diego):** `public.perfiles(id uuid)` ligada a `auth.users`; `auth.uid()` como identidad; `public.es_moderador() returns boolean` (basada en `auth.uid()`).
- **M4/M5:** `public.simulaciones(id uuid, autor_id uuid → perfiles, estado text)` y que el estado `'publicada'` (RF-17) sea el que hace visible una simulación. M4 debe llamar `registrar_vista` al abrir la página de simulación.
- **M2:** sin dependencia directa.

5. Modelo de datos afectado (tablas nuevas)

- `favoritos(usuario_id, simulacion_id, creado_en)` — PK `(usuario_id, simulacion_id)`; índice `(usuario_id, creado_en desc)`.
- `valoraciones(usuario_id, simulacion_id, puntaje smallint 1..5, creado_en, actualizado_en)` — PK `(usuario_id, simulacion_id)`; índice por `simulacion_id`; trigger que impide al autor valorar lo suyo.
- `comentarios(id uuid, usuario_id, simulacion_id, contenido, creado_en, editado_en, eliminado_en)` — índice `(simulacion_id, creado_en desc)`.
- `vistas_simulacion(id bigint, simulacion_id, visitante_hash char(64), dia date)` — UNIQUE `(simulacion_id, visitante_hash, dia)`.
- Todas las FK a `perfiles` y `simulaciones` son `ON DELETE CASCADE`.

Seguridad (RLS activo en las 4 tablas): cada usuario solo lee, crea y borra sus favoritos y valoraciones y solo sobre simulaciones publicadas; comentarios públicos de simulaciones publicadas, solo el autor edita, autor o moderador ocultan; `vistas_simulacion` sin acceso directo (solo vía `registrar_vista`). El rol `anon` no accede a favoritos ni a filas de valoraciones. No se usa ninguna clave privilegiada en el navegador; las funciones `SECURITY DEFINER` fijan `search_path`.

6. Cómo probarlo

Requisitos: PostgreSQL 14+ y `psql` en una máquina local (no se necesita Supabase para estas pruebas).

```
cd m6-db
bash tests/run_tests.sh      # como un usuario con permisos para createdb/dropdb
```

El script crea la BD `m6_test`, aplica los stubs, las 4 migraciones, 51 pruebas (23 + 28), las reversas y vuelve a aplicar todo.
En Supabase real: aplicar los archivos de `supabase/migrations/` en orden (SQL Editor o CLI) cuando existan `perfiles`, `simulaciones` y `es_moderador()`. Para revertir, ejecutar los `.down.sql` en orden inverso.

Resultado real (ejecutado el 26-oct-2026 en PostgreSQL 16.15, Ubuntu): 51 de 51 pruebas pasaron; aplicar → revertir → reaplicar funcionó sin errores.

Durante las pruebas se detectó y corrigió un fallo real: el borrado lógico de comentarios fallaba por RLS hasta añadir la política `comentarios_select_autor_o_moderador`.

7. Problemas conocidos y pendientes

- **No probado contra Supabase real**, ni con las tablas reales de M1/M4/M5 (se usaron stubs en `tests/00_stubs_provisionales.sql`).
- La prueba V2 (puntaje 0 y 6) usa una simulación no publicada: el rechazo puede venir de RLS y no del `CHECK`. Repetir con una simulación publicada.
- No hay archivo de datos de prueba para desarrollo (`0005`): los datos viven dentro de los tests.
- La estructura del autor visible en comentarios (nombre) depende de `perfiles` (M1); no hay vista pública de perfiles.
- Sin pruebas de carga ni de concurrencia. El conteo de vistas por día usa UTC.
- Falta la parte de interfaz (diseño, pantallas, RNF-04).

8. Cambios solicitados a otros módulos

- **M1:** confirmar `perfiles(id uuid)`, `es_moderador()` y cómo se asignan roles.
- **M4/M5:** confirmar columnas `id`, `autor_id`, `estado` y el valor `'publicada'`; M4 llama a `registrar_vista`.
- **Todos:** confirmar IDs `uuid`.

9. Contexto para el siguiente integrante

Ver `CONTEXTO_PARA_ASISTENTES.md`.